

YingMusic-Singer：具有灵活歌词操控和无标注旋律引导的可控歌声合成

Chunbo Hao^{1,2}, Junjie Zheng², Guobin Ma¹, Yuepeng Jiang¹, Huakang Chen¹, Wenjie Tian¹, Gongyu Chen², Zihao Chen², Lei Xie^{1,*}

¹ 西北工业大学计算机学院音频、语音与语言处理组 (ASLP@NPU), 中国 ² 巨人网络AI Lab, 中国 cbhao@mail.nwpu.edu.cn, zhengjunjie@ztgame.com, lxie@nwpu.edu.cn

摘要

在保持旋律一致性的同时，用修改后的歌词重新生成歌声仍然具有挑战性，因为现有方法要么可控性有限，要么需要费力的人工对齐。我们提出了 YingMusic-Singer，一个完全基于扩散的模型，能够实现旋律可控的歌声合成和灵活的歌词操控。该模型接收三个输入：一个可选的音色参考、一个提供旋律的歌声片段，以及修改后的歌词，无需人工对齐。通过课程学习和组相对策略优化（GRPO）训练，YingMusic-Singer 在旋律保持和歌词遵循方面均优于 Vevo2——最可比的、支持无人工对齐旋律控制的基线。我们还引入了 LyricEditBench，这是首个用于旋律保持歌词修改评估的基准。代码、权重、基准和演示可在 <https://github.com/ASLP-lab/YingMusic-Singer-Plus> 公开获取。

索引词：歌声合成，歌词编辑，强化学习，扩散模型

1. 引言

歌声合成（SVS）旨在从乐谱、歌词和音色参考生成类人歌声。现代系统 [1, 2, 3, 4, 5, 6] 实现了高保真合成，但大多数依赖精确标注的配对数据，将每个音素与确切的音高轮廓和时长关联。虽然这种细粒度控制对于专业音乐制作不可或缺，但准备这些标注为一个日益重要的用例——歌声编辑——制造了难以逾越的障碍，该用例在保持原始旋律和节奏结构的同时，用修改后的歌词重新生成现有歌声。这一能力对于歌曲改编、个性化翻唱生成、人声编曲快速原型制作以及多语言歌曲本地化都极为理想。

现有的编辑范式分为两类。第一类采用上下文学习策略，掩码待编辑区域并根据周围上下文和目标歌词重新生成 [7, 8]。虽然方便，但该方法局限于局部片段，且旋律控制有限。第二类在实践中广泛采用，依赖商业 SVS 模型如 Synthesizer V¹ 和 ACE Studio，用户首先需要将原始人声转录为 MIDI，然后将修改后的歌词分配到音符和时长上，再重新合成音频。该流程提供了强大的可控性，但需要人工操作，且对于跨语言翻译等任务，复杂性增加。总之，现有方法要么依赖周围上下文来恢复旋律，要么需要将词级时间戳与旋律信息进行人工对齐，限制了其灵活性和可扩展性。最近的一些努力应对了这些挑战。Vevo2 [9] 实现了旋律可控生成，但存在可懂度降低和旋律遵循不佳的问题。SoulX-Singer [10] 支持将现有歌声作为旋律输入，但仍需要人工对齐词级时间戳，核心负担未解决。虽然这些近期努力缩小了差距，但它们要么性能次优，要么仍需要人工对齐。

为应对这些挑战，我们提出了 YingMusic-Singer，一个用于旋律保持歌词编辑的完全基于扩散的模型。首先，YingMusic-Singer 引入了一种简化的编辑范式，仅从三个输入合成歌声，即一个可选的音色参考片段、一个提供目标旋律的歌声片段，以及相应的修改后歌词，无需人工对齐或精确标注。其次，为缓解歌声数据规模小和演唱技巧复杂导致的音素泛化有限，并解决忠实歌词再现与旋律遵循之间的固有权衡，我们采用课程训练策略结合组相对策略优化（GRPO）[11, 12]，使其在两个维度上同时具备强大性能。第三，我们基于 GTSinger [13] 构建了 LyricEditBench，这是首个在匹配旋律条件下进行歌词修改评估的基准，涵盖六种常见编辑场景，并采用平衡采样以进行公平全面的比较。实验表明，YingMusic-Singer 在旋律保持和歌词遵循方面均优于 Vevo2 [9]——目前可比的免对齐替代方案。我们公开发布模型权重、推理代码和 LyricEditBench，以促进进一步研究并推动歌声编辑的实际应用。

2. 方法

2.1. 架构概览

如图1所示，YingMusic-Singer 从三个输入生成 44.1kHz 的歌声：一个可选的音色参考、一个提供旋律的歌声片段，以及相应的修改后歌词。它包括：（1）一个遵循 Stable Audio 2 [14] 的变分自编码器（VAE），其编码器 $\mathcal{E}$ 将立体声 44.1kHz 歌声波形 $\mathbf{x} \in \mathbb{R}^{T \times 2}$ 下采样 2048 倍为 $\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{T \times D}$ ，其解码器 $\mathcal{D}$ 在推理时重建高保真音频 $\hat{\mathbf{x}} = \mathcal{D}(\mathbf{z})$ ；（2）一个建立在预训练 MIDI 提取模型编码器之上的旋律提取器，其中间表示自然捕获解耦的旋律信息。它产生 $\mathbf{h} = \mathcal{M}(\mathbf{M}) \in \mathbb{R}^{L \times D_m}$ ，然后时间插值为 $\hat{\mathbf{h}} \in \mathbb{R}^{T' \times D_m}$ 以匹配 VAE 隐变量帧率；（3）一个 IPA Tokenizer，将中文和英文歌词转换为统一的离散音素序列。为确保模型正确区分提示区域和生成区域，避免边界处的音素遗漏或重复，我们采用遵循 DiffRhythm [15] 的句子级对齐。每个歌词句子转换为 IPA 子序列，并放置在长度为 T' 的填充帧级序列中对应的起始帧处。对齐后的序列通过可学习嵌入层产生 $\mathbf{e} \in \mathbb{R}^{T' \times D_e}$ 。在推理期间，提示歌词放在序列开头，目标歌词放在掩码区域开头，无需用户提供时间戳标注；以及（4）一个遵循 F5-TTS [16] 的基于 DiT 的 CFM 主干。

在训练期间，一定比例 γ 的 VAE 隐变量帧被随机掩码作为合成目标，未掩码部分作为音色上下文。令 $\mathbf{z}_{\text{cts}}$ 表示未掩码的 VAE 隐变量（在掩码区域零填充）。条件 $\mathbf{c} = [\mathbf{h}; \mathbf{e}; \mathbf{z}_{\text{cts}}]$ 与带噪隐变量 $\mathbf{z}_t = (1 - t)\mathbf{z}_0 + t\mathbf{z}_1$ 沿通道维度拼接，并馈入 CFM，后者通过以下方式学习速度场：

$$\mathcal{L}_{\text{MSE}} = \mathbb{E}_{t, \mathbf{z}_0, \mathbf{z}_1, \mathbf{c}} \|v_{\theta}(\mathbf{z}_t, t, \mathbf{c}) - (\mathbf{z}_1 - \mathbf{z}_0)\|^2. \quad (1)$$

2.2. 课程训练

为缓解歌声数据规模小和演唱技巧复杂导致的音素泛化有限，YingMusic-Singer 首先进行无旋律条件的 TTS 预训练。随后的歌声监督微调（SFT）阶段有两个阶段。阶段1在歌声数据上启用句子级对齐，使模型适应歌声领域。阶段2激活旋律条件，并引入中心核对齐（CKA）损失以强制旋律遵循。给定预测的 v_θ 和旋律 $\tilde{\mathbf{h}}_t$ ，CKA 通过 Gram 矩阵 $\mathbf{K} = v_\theta v_\theta^T$ 和 $\mathbf{L} = \tilde{\mathbf{h}}\tilde{\mathbf{h}}^T$ 测量它们的对齐。

$$\mathcal{L}_{\text{CKA}} = 1 - \frac{\|\mathbf{K}^T \mathbf{L}\|_F^2}{\|\mathbf{K}^T \mathbf{K}\|_F \|\mathbf{L}^T \mathbf{L}\|_F}, \quad (2)$$

阶段2的总损失为 $\mathcal{L}_{\text{SFT}} = \mathcal{L}_{\text{MSE}} + \lambda \mathcal{L}_{\text{CKA}}$ 。

2.3. 组相对策略优化

虽然课程训练取得了高性能，但 SFT 阶段2同时降低了 PER，暴露了持续权衡。 $\mathcal{L}_{\text{MSE}}$ 针对整体隐变量重建，无法隔离特定缺陷进行针对性优化，而调整 $\mathcal{L}_{\text{CKA}}$ 中的 λ 仅改变平衡而不解决权衡本身。此外，大规模歌声数据中不可避免的噪声，如背景人声和质量伪影，将模型性能限制在数据集上限。为克服这些限制，强化学习（RL）变得至关重要。PPO 需要价值网络，训练成本高。离线方法如 DPO 遭受分布偏移，因为当策略改进时，预先收集的偏好数据变得过时。GRPO [11, 12] 在线运行，从组内奖励统计中估计基线，消除了价值网络，同时保持高效和稳定。

遵循近期努力 [17, 18, 19]，我们将确定性 ODE 轨迹转换为 SDE，但将随机步限制在有界窗口内，确保精确的优势归因于探索步。为防止坍塌到单一奖励维度，我们联合使用 M 个奖励模型，并计算每个样本的优势为

$$A^i = \sum_{k=1}^M w_k \frac{R_k^i - \text{mean}(\{R_k^i\})}{\text{std}(\{R_k^i\})}, \quad (3)$$

最终的 GRPO 损失为

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|S|} \sum_{t \in S} (-\mathcal{L}_{\text{clip}} + \beta D_{\text{KL}}), \quad (4)$$

其中 $\mathcal{L}_{\text{clip}}$ 是当前与旧策略似然比的裁剪代理目标， D_{KL} 将当前策略正则化 toward 参考， G 是组大小， S 是 SDE 采样步集合， β 是 KL 正则化强度。

2.4. LyricEditBench

我们基于 GTSinger [13] 构建 LyricEditBench，移除所有 Paired Speech Group 内容，通过 MD5 哈希去重音频，并排除超过 15 秒的片段。DeepSeek V3.2 [20] 然后为表1中的六种修改类型分别生成修改后的歌词。给定原始歌词和修改指令，LLM 产生修订版本，不符合要求的输出被丢弃，得到 11,535 个有效样本。样本按歌手性别（男/女）和语言（中文/英文）分为四类，然后按修改类型组织。对于每种组合，我们为每种演唱技巧（覆盖 GTSinger 中的六种技巧）选择 30 个样本，为无技巧类别选择 120 个，结果为每种修改类型每类 300 个，总计 7,200 个测试实例。对于每个实例，从剩余音频池中随机抽取一个不超过 15 秒的音色提示，因此每个 LyricEditBench 实例包含一个旋律参考片段、一个音色提示和相应的修改后歌词。

表1: LyricEditBench 中歌词修改的任务类型。

缩写	任务类型	描述
PSub	部分替换	替换部分词语
FSub	完全替换	完全重写歌曲
Del	删除	删除一些词语
Ins	插入	插入一些词语
Trans	翻译	中英互译
Mix	语码混合	混合语言歌词

3. 实验设置

数据集。 Emilia [21] 的中文和英文子集用于 TTS 预训练。对于歌声 SFT，内部授权的音乐曲目由 SongFormer [22] 处理，以分割结构边界并标记功能类别，丢弃非人声片段。然后使用 Mel-band RoFormer [23] 分离人声干声。我们保留 2 到 30 秒的片段，在句子边界处分割较长的片段，最终获得 33,562.6 小时的歌声数据。GRPO 数据集通过三个标准过滤 SFT 数据构建：ASR 转录验证，仅保留词错误率低于 5% 的片段；通过 pyannote [24, 25] 进行说话人日志，仅保留单说话人片段；以及 DNSMOS P.808 质量分数 [26, 27] 阈值 3.5。这产生了约 20,240 个精选片段，中英文内容平衡。所提出的 LyricEditBench 中的测试集严格排除在训练之外。

评估指标。我们用四个客观和两个主观指标评估模型。对于客观评估，音素错误率（PER）衡量音素级可懂度，因为歌声比语音具有更低的语义密度、更少的上下文线索和更大的发音变化。中文和英文片段均由歌声训练的 Qwen3-ASR-1.7B [28] 转录，并转换为音素序列，去除声调标记。说话人相似度（SIM）遵循 F5-TTS [16]，使用基于 WavLM-large 的验证模型提取说话人嵌入并计算余弦相似度。F0 皮尔逊相关（F0-CORR）通过使用 RMVPE [29] 计算生成和参考片段 F0 轮廓之间的逐帧皮尔逊相关来测量旋律遵循。人声分数（VS）采用 VocalVerse2 [30] 作为与人类感知偏好对齐的学习度量。对于主观评估，30 名听众对跨任务类型和语言均匀采样的 120 个样本在两个维度上评分：自然度平均意见分数（N-MOS）用于整体感知质量和自然度，旋律平均意见分数（M-MOS）用于对参考旋律的忠实度，均为 5 分制。

实现细节。我们采用 Stable Audio 2 [14] 的 VAE ($D = 64$)、SOME 的编码器作为旋律提取器 ($D_m = 128$ ，时间 dropout 0.1)，以及遵循 F5-TTS [16] 的 DiT 主干（22 层，16 头，隐藏维度 1024， $D_e = 512$ ）。完整系统约有 727.3M 参数（453.6M CFM，156.1M VAE，117.6M 旋律提取器），在 $8 \times A800$ 80GB GPU 上使用 DDP 和 bf16 训练。在所有阶段，70% – 100% 的隐变量帧被随机掩码。TTS 预训练运行 1M 步（批次时长 1.268h，学习率 $1e-4$ ）。歌声 SFT 阶段1禁用旋律条件 240K 步；阶段2启用 170K 步（ λ 在前 2K 步从 0.3 衰减到 0.01；批次时长 1.69h，学习率 $2.5e-5$ ）。对于 GRPO， $G = 8$ 个候选由 $M = 4$ 个等权重奖励模型评分（SDE 噪声 $a = 0.8$ ，窗口 $\mathcal{W}$ 其中 $w_{\min} = 1$ ， $w_s = 8$ ， $\epsilon_u = 0.01$ ， $\epsilon_l = 0.002$ ， $\beta = 1$ ），优化 1.2K 步（批次大小 6，学习率 $7e-6$ ），无 CFG。推理使用 32 个 ODE 步，CFG 尺度 3。

4. 实验结果

4.1. 主要结果

我们与 Vevo2 [9] 进行比较，后者是一个基于 token 的自回归模型，具有解耦的音色和旋律控制。对于 Sing Edit（自音色），音色和旋律参考共享同一片段；对于 Melody Control（跨音色），音色参考相应替换。Vevo2 是最直接的基线，因为它在可比输入条件下支持相同功能。其他近期系统在根本不同的范式下运行。上下文学习方法需要周围音频上下文和人工对齐的编辑边界，将其限制在局部片段编辑。SoulX-Singer 依赖精确的字符级时间戳；通过 ASR 获得此类标注会引入非平凡的字符数不匹配，而来自精选数据集的真实时间戳会赋予在实际中不存在的非现实优势。

如表2所示，YingMusic-Singer 在 Melody Control 和 Sing Edit 两种设置下，在所有六种修改类型中，在 PER、F0-CORR 和 VS 上均持续优于 Vevo2，展示了对参考旋律和修改后歌词的强遵循。可懂度差距在 Trans 和 Mix 任务上最为明显，表明在保持旋律的同时重建显著不同的音素序列本质上是具有挑战性的。注意 Mix 任务上的 PER 可能因 ASR 对语码转换话语的幻觉而进一步膨胀。Vevo2 不完整的旋律解耦倾向于降低可懂度并引入幻觉，而 YingMusic-Singer 的统一 IPA tokenization 和基于 GRPO 的歌词遵循优化即使在这些极端条件下也保持鲁棒性。F0-CORR 进一步区分两个系统：受益于 CKA 对齐和 GRPO，YingMusic-Singer 在所有任务和语言中保持持续高相关，而 Vevo2 波动较大，表明旋律控制不够稳健。对于 SIM，Vevo2 受益于其多阶段架构，其中自回归 LLM 处理旋律和内容生成，而专用 CFM 专注于音色重建，有效简化了说话人建模。YingMusic-Singer 则采用单阶段 CFM 联合建模所有因素，优先考虑架构简单性以便实际部署。然而，Vevo2 更高的 SIM 以降低 PER 和 F0-CORR 为代价。在实践中，忠实渲染修改后的歌词同时保持原始旋律结构仍然是歌词编辑中的首要关注点。如表3所示，YingMusic-Singer 在两种任务和语言中均持续获得比 Vevo2 更高的 N-MOS 和 M-MOS。强主观分数也表明 GRPO 优化没有过拟合奖励模型，而是泛化到人类感知。Vevo2 总体评分较低且方差明显更高，听众报告其输出中有可感知的伪影，如不忠实的歌词渲染和旋律错位，表明生成质量不够稳健。

除了模型比较，跨六种编辑类型、两种语言以及客观和主观指标的评估广度进一步验证了 LyricEditBench 作为旋律保持歌词编辑综合基准的有效性，支持未来关于歌曲改编、翻唱生成和跨语言人声编曲的研究。

表2：与基线模型在 LyricEditBench 上跨表1任务类型和语言的比较。指标 (M)：P: PER, S: SIM, F: F0-CORR, V: VS 详见第3节。最佳结果以粗体显示。

任务	模型	M	中文 PSub	中文 FSub	中文 Del	中文 Ins	中文 Trans	中文 Mix	英文 PSub	英文 FSub	英文 Del	英文 Ins	英文 Trans	英文 Mix
Melody Control (跨音 色)	Vevo2 [9]	P↓	0.1378	0.1462	0.1545	0.1872	0.4409	0.4757	0.3352	0.3481	0.3812	0.3135	0.8019	0.5132
		S↑	0.6462	0.6550	0.6457	0.6551	0.6115	0.6490	0.6357	0.6161	0.6277	0.6359	0.6237	0.6325
		F↑	0.8471	0.8188	0.8345	0.8552	0.7678	0.8526	0.8794	0.8409	0.8924	0.8888	0.8776	0.8927
		V↑	1.3578	1.3784	1.3491	1.1346	1.3061	1.4208	1.0340	1.1217	1.0476	0.9610	1.0281	1.0925
Sing Edit (自 音色)	Ours	P↓	0.0192	0.0197	0.0458	0.0208	0.0881	0.1563	0.0685	0.0692	0.1053	0.0716	0.0413	0.2668
		S↑	0.6543	0.6561	0.6489	0.6552	0.5791	0.6395	0.6078	0.5914	0.6001	0.5889	0.5982	0.6076
		F↑	0.9364	0.9428	0.9351	0.9381	0.9378	0.9352	0.9355	0.9279	0.9309	0.9315	0.9290	0.9389

任务	模型	M	中文 PSub	中文 FSub	中文 Del	中文 Ins	中文 Trans	中文 Mix	英文 PSub	英文 FSub	英文 Del	英文 Ins	英文 Trans	英文 Mix
Sing Edit (自 音色)	Vevo2 [9]	V↑	2.0779	2.1419	2.1219	1.9887	1.9372	2.1002	1.5054	1.5418	1.6081	1.4036	1.5769	1.5060
		P↓	0.1290	0.1303	0.1596	0.1810	0.4111	0.4659	0.3414	0.3538	0.3531	0.2944	0.7680	0.4951
		S↑	0.7875	0.7729	0.8269	0.8324	0.7252	0.8015	0.7971	0.7729	0.8183	0.8378	0.7563	0.8346
	Ours	F↑	0.8858	0.8805	0.8969	0.9115	0.8456	0.9023	0.9258	0.9278	0.9365	0.9415	0.9137	0.9465
		V↑	1.4860	1.5100	1.5094	1.2935	1.3535	1.4377	1.0910	1.1110	1.1682	1.1156	1.1178	1.1453
		P↓	0.0214	0.0186	0.0946	0.0426	0.1009	0.1903	0.0906	0.0782	0.1700	0.1070	0.0538	0.2946
		S↑	0.7622	0.7392	0.7874	0.8028	0.6539	0.7564	0.7398	0.7105	0.7764	0.7714	0.6918	0.7708
		F↑	0.9615	0.9587	0.9628	0.9642	0.9542	0.9607	0.9610	0.9563	0.9675	0.9660	0.9498	0.9668
		V↑	1.9761	2.0345	1.8837	1.7689	1.9371	1.9283	1.4448	1.4086	1.3820	1.2553	1.4788	1.3464

表3：LyricEditBench 上的主观评估。N-MOS 和 M-MOS 分别表示自然度和旋律遵循。

任务	模型	中文 N↑	中文 M↑	英文 N↑	英文 M↑
Melody Control	Vevo2 [9]	4.25±0.06	4.28±0.05	4.31±0.05	4.31±0.04
	Ours	4.31±0.04	4.44±0.04	4.36±0.05	4.51±0.03
Sing Edit	Vevo2 [9]	4.48±0.05	4.41±0.05	4.44±0.04	4.50±0.04
	Ours	4.52±0.04	4.55±0.04	4.55±0.04	4.58±0.03

4.2. 消融研究

如表4所示，课程学习流程在每个阶段引入了清晰可分离的改进。TTS 预训练建立了发音先验，但缺乏歌声能力（F0-CORR 接近零），当歌声片段作为 ICL 提示时 PER 显著退化，表明领域适应不佳。SFT 阶段1大幅改善所有指标，实现最低 PER，因为模型适应歌声领域，同时从 ICL 提示自由生成旋律，绕过了显式旋律对齐的艰巨任务。Sing Edit 下的 F0-CORR 略有改善，显示仅从上下文部分捕获旋律，但忠实再现仍需要显式引导。SFT 阶段2激活旋律提取器，将 F0-CORR 提高到 0.92 以上，但 PER 增加，反映了同时保持旋律保真度和歌词忠实度的困难。GRPO 通过恢复 PER 同时进一步改善 F0-CORR 和 VS（SIM 不变）解决了这一权衡，确认基于奖励的优化联合增强了所有维度。

在 SFT 阶段2中，引入 CKA 进一步改善了旋律遵循，反映在 F0-CORR 增益上。对于扰动消融（w/o Dist），我们移除了应用于旋律隐变量 $\hat{h}$ 的时间 dropout。这导致严重的可懂度退化，因为未扰动的隐变量保留了残余语义信息，模型利用这些信息绕过真正的歌词生成。时间 dropout 消除了这种泄漏，迫使依赖抽象旋律轮廓，并在允许自由生成修改后歌词的同时保持韵律结构。

表4：LyricEditBench 上的消融研究。最佳结果加粗，次佳结果下划线。

语言	变体	Melody Control P↓	Melody Control S↑	Melody Control F↑	Sing Edit P↓	Sing Edit S↑	Sing Edit F↑
中文	TTS Pretrain	0.41	0.57	0.01	0.50	0.37	0.59
	SFT Phase1	0.05	0.68	0.03	1.55	0.05	0.73
	SFT Phase2	0.08	0.63	0.92	1.57	0.11	0.75
	-w/o CKA	0.08	0.64	0.91	1.57	0.12	0.75
	Full Model-w/o Dist	0.45	0.63	0.94	1.42	0.46	0.79
	Full Model	0.06	0.64	0.94	2.06	0.08	0.75
英文	TTS Pretrain	0.46	0.54	0.01	0.49	0.43	0.56
	SFT Phase1	0.10	0.65	0.03	1.07	0.11	0.73

语言	变体	Melody Control P↓	Melody Control S↑	Melody Control F↑	Sing Edit P↓	Sing Edit S↑	Sing Edit F↑
	SFT Phase2	0.14	0.60	0.92	1.17	0.19	0.75
	-w/o CKA	0.14	0.60	0.90	1.19	0.20	0.75
	-w/o Dist	0.48	0.58	0.95	1.00	0.49	0.78
	Full Model	0.10	0.60	0.93	1.52	0.13	0.74

5. 结论

我们提出了 YingMusic-Singer，一个旋律可控的歌声编辑模型，从音色参考、提供旋律的歌声片段和修改后的歌词合成，无需人工对齐。通过课程训练和基于 GRPO 的强化学习，YingMusic-Singer 实现了.....

6. 生成式 AI 使用披露

生成式 AI 工具仅用于语言润色，在方法、实验、解释或科学结果产生中不发挥任何作用。作者对本手稿中的所有内容承担全部智力责任。
(参考文献已按用户要求省略。)